EXPERIMENT 6: ADVERTISING SALES PREDICTION AND K-MEANS CLUSTERING

AIM:

To analyze the Advertising Budget and Sales dataset by exploring the data, splitting it into training and testing sets, building and comparing Linear Regression, Polynomial Regression, and Support Vector Regression models, and applying K-Means clustering to identify groups in the advertising data.

ALGORITHM:

1. Import the required libraries, including Pandas, NumPy, Seaborn, Matplotlib, train_test_split, LinearRegression, PolynomialFeatures, make_pipeline, SVR, KMeans, and error metrics.
2. Load the Advertising Budget and Sales CSV dataset and display the first five rows.
3. Check missing values and inspect the dataset data types.
4. Select TV, Radio, and Newspaper advertising budgets as input features (X) and Sales as the target (y).
5. Visualize the relationships and distributions of the advertising budget features using a pair plot.
6. Split the data into training and testing sets using test_size=0.2 and random_state=42.
7. Train a Linear Regression model and generate predictions for the test data.
8. Train a Polynomial Regression model using PolynomialFeatures with degree 2 and generate predictions.
9. Train a Support Vector Regression model using an RBF kernel and generate predictions.
10. Evaluate the models using Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

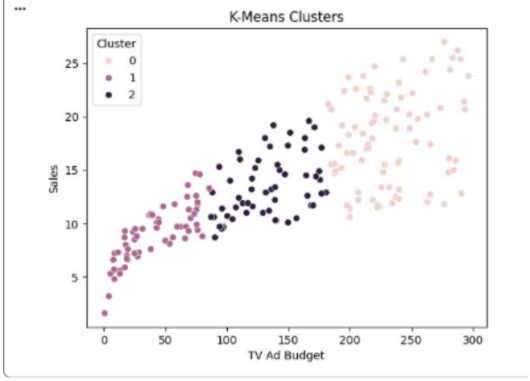
11. Compare the evaluation metrics for Linear, Polynomial, and SVR models using a bar chart.
12. Apply K-Means clustering with three clusters to the advertising dataset and add the cluster labels to the DataFrame.
13. Visualize the clusters using a scatter plot of TV advertising budget against Sales.

```
# EXERCISE-3 MUTHURAJ R (241801176)
# 9. Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)
print(df[["TV Ad Budget ($)", "Radio Ad Budget ($)", "Newspaper Ad Budget ($)", "Sales ($)", "Cluster"]].head())
```

```
   TV Ad Budget ($)  Radio Ad Budget ($)  Newspaper Ad Budget ($)  Sales ($)  \
0             230.1                 37.8                     69.2       22.1
1              44.5                 39.3                     45.1       10.4
2              17.2                 45.9                     69.3        9.3
3             151.5                 41.3                     58.5       18.5
4             180.8                 10.8                     58.4       12.9

   Cluster
0        0
1        1
2        1
3        2
4        2
```

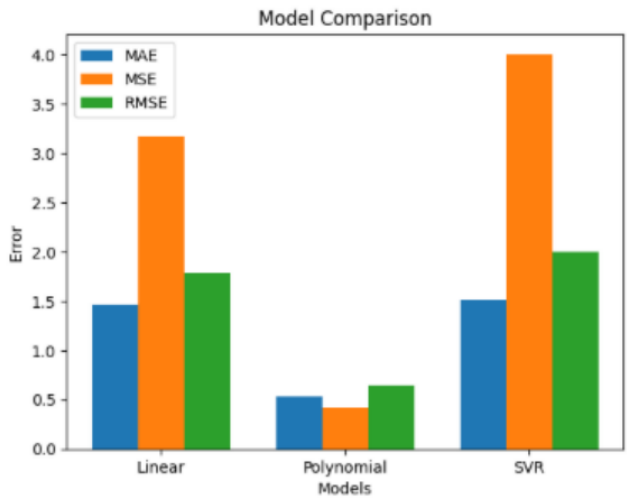
```
# EXERCISE-3 MUTHURAJ R (241801176)
# 10. Visualize Clusters
sns.scatterplot(
    x=df["TV Ad Budget ($)"],
    y=df["Sales ($)"],
    hue=df["Cluster"]
)
plt.title("K-Means Clusters")
plt.xlabel("TV Ad Budget")
plt.ylabel("Sales")
plt.show()
```



```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 5. Train Polynomial Regression
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X_train, y_train)
poly_pred = poly_model.predict(X_test)
print("Polynomial Regression Predictions")
print(poly_pred)
```

```
Polynomial Regression Predictions
[17.25443578 22.7193321  20.42799436  7.542709   24.37103037 12.55927164
 22.80784986  8.34371358 12.0586023  15.69024565  7.89166367  8.27082715
 11.86055971  6.16591094 10.57664393 12.3628954   6.74390963 16.65082111
 10.68661722 19.03888161 20.15223945 13.1137814   9.56273868 22.10675225
  8.96725241  7.7794437  22.40745151 12.72065973 10.25529986  6.22368636
 11.64677688 10.22431946 23.39763949  9.17403232 15.36143449 21.05675814
 10.9795286  20.23370753 11.85153256  6.58779915]
```

```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 8. Model Comparison
models = ["Linear", "Polynomial", "SVR"]
mae = [linear_mae, poly_mae, svr_mae]
mse = [linear_mse, poly_mse, svr_mse]
rmse = [linear_rmse, poly_rmse, svr_rmse]
x = np.arange(len(models))
width = 0.25
plt.bar(x - width, mae, width, label="MAE")
plt.bar(x, mse, width, label="MSE")
plt.bar(x + width, rmse, width, label="RMSE")
plt.xticks(x, models)
plt.xlabel("Models")
plt.ylabel("Error")
plt.title("Model Comparison")
plt.legend()
plt.show()
```



```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 7. Evaluate Models
linear_mae = mean_absolute_error(y_test, linear_pred)
linear_mse = mean_squared_error(y_test, linear_pred)
linear_rmse = np.sqrt(linear_mse)
poly_mae = mean_absolute_error(y_test, poly_pred)
poly_mse = mean_squared_error(y_test, poly_pred)
poly_rmse = np.sqrt(poly_mse)
svr_mae = mean_absolute_error(y_test, svr_pred)
svr_mse = mean_squared_error(y_test, svr_pred)
svr_rmse = np.sqrt(svr_mse)
print("Linear Regression")
print("MAE:", linear_mae)
print("MSE:", linear_mse)
print("RMSE:", linear_rmse)
print("\nPolynomial Regression")
print("MAE:", poly_mae)
print("MSE:", poly_mse)
print("RMSE:", poly_rmse)
print("\nSupport Vector Regression")
print("MAE:", svr_mae)
print("MSE:", svr_mse)
print("RMSE:", svr_rmse)
```

```
Linear Regression
MAE: 1.4607567168117603
MSE: 3.1740973539761033
RMSE: 1.78159966153345

Polynomial Regression
MAE: 0.5261794444043838
MSE: 0.41291022853790765
RMSE: 0.6425809120553673

Support Vector Regression
MAE: 1.514357586594181
MSE: 4.00604540443217
RMSE: 2.0015107804936174
```

```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 6. Train Support Vector Regression
svr_model = SVR(kernel="rbf")
svr_model.fit(X_train, y_train)
svr_pred = svr_model.predict(X_test)
print("SVR Predictions")
print(svr_pred)
```

```
SVR Predictions
[16.82433682 19.737857   18.51883217  8.49757937 19.14449648 12.35892516
 19.35370785  8.670194   14.26079367 15.69961405  8.67459241  8.63883765
 16.10338892  7.86941414 11.80945227 13.73449615  7.82912158 16.07178231
 10.8096626  17.39870345 19.02145965 12.85124102  9.98938778 19.67786747
  9.38640699  8.22363571 19.35662438 12.9459954  10.6084116   7.91787133
 13.69013745 10.3362837  20.05661258  9.88723936 17.12640815 17.96277376
 11.10345511 18.94964504 12.02692034  7.21381614]
```

```python
# EXERCISE-3
# MUTHURAJ R-(241801176)
# 1. Import Libraries and Load Advertising Dataset
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("Advertising Budget and Sales.csv")
print("Dataset Loaded Successfully")
print(df.head())
```

```
Dataset Loaded Successfully
   Unnamed: 0  TV Ad Budget ($)  Radio Ad Budget ($)  Newspaper Ad Budget ($)  \
0           1             230.1                 37.8                     69.2
1           2              44.5                 39.3                     45.1
2           3              17.2                 45.9                     69.3
3           4             151.5                 41.3                     58.5
4           5             180.8                 10.8                     58.4

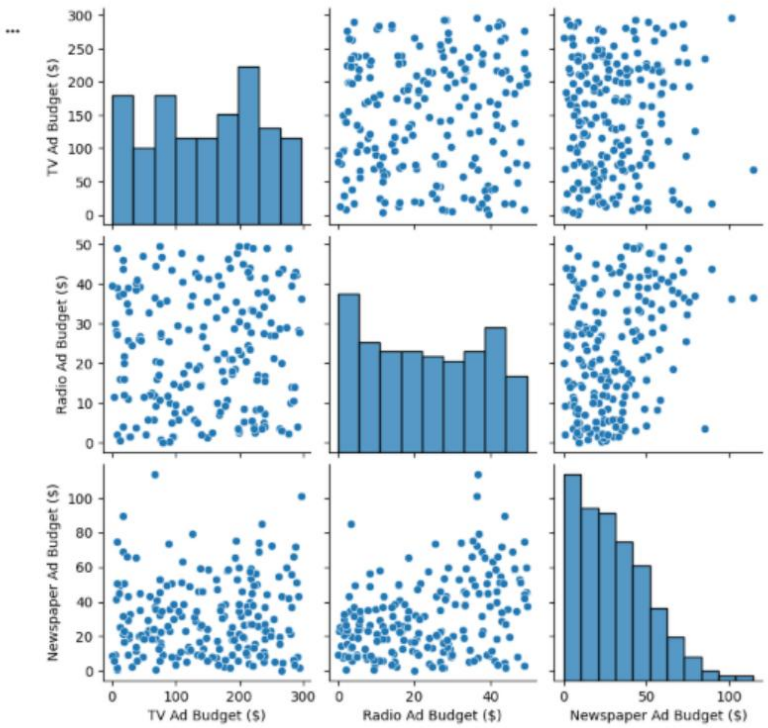
   Sales ($)
0       22.1
1       10.4
2        9.3
3       18.5
4       12.9
```

```python
# EXERCISE-3
# MUTHURAJ R (241801176)
# 4. Train Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
print("Linear Regression Predictions")
print(linear_pred)
```

```
Linear Regression Predictions
[16.4080242  20.88988209 21.55384318 10.60850256 22.11237326 13.10559172
 21.05719192  7.46101034 13.60634581 15.15506967  9.04831992  6.65328312
 14.34554487  8.90349333  9.68959028 12.16494386  8.73628397 16.26507258
 10.27759582 18.83109103 19.56036653 13.25103464 12.33620695 21.30695132
  7.82740305  5.80957448 20.75753231 11.98138077  9.18349576  8.5066991
 12.46646769 10.00337695 21.3876709  12.24966368 18.26661538 20.13766267
 14.05514005 20.85411186 11.0174441   4.56899622]
```

```python
# EXERCISE-3
# MUTHURAJ R (241801176)
# 3. Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (160, 3)
Testing Data: (40, 3)
```




```
# EXERCISE-3
# MUTHURAJ R (241801176)
# 2. Explore and Preprocess Data
print("Null Values")
print(df.isnull().sum())
print("\nData Types")
print(df.dtypes)
X = df[["TV Ad Budget ($)", "Radio Ad Budget ($)", "Newspaper Ad Budget ($)"]]
y = df["Sales ($)"]
sns.pairplot(X)
plt.show()
```

```
Null Values
Unnamed: 0                  0
TV Ad Budget ($)            0
Radio Ad Budget ($)         0
Newspaper Ad Budget ($)     0
Sales ($)                   0
dtype: int64

Data Types
Unnamed: 0                   int64
TV Ad Budget ($)           float64
Radio Ad Budget ($)        float64
Newspaper Ad Budget ($)    float64
Sales ($)                  float64
dtype: object
```

RESULT:

The Advertising Budget and Sales dataset was loaded and explored, with no missing values reported in the displayed output. Linear Regression, degree-2 Polynomial Regression, and Support Vector Regression were trained and their predictions generated. The models were evaluated and compared using MAE, MSE, and RMSE. K-Means clustering with three clusters was also applied, and the resulting groups were visualized by plotting TV advertising budget against Sales.